

Simple 4-Bit DOWN Counter

Introduction:

A simple 4-bit down-counter is a fundamental digital circuit capable of counting and storing numerical values represented by four binary digits. It is designed to decrement (count down). This means it can count from Binary 4'b1111 (decimal 15) down to 4'b0000 (decimal 0), providing 16 distinct states in its counting cycle. It is designed using flip-flops, the counter decreases its stored binary count with each incoming clock pulse.

Verilog Code Description:

Verilog module named `down_counter_4bit` which implements a 4-bit simple down counter. This module has three ports: clock input (`clk`), input reset (asynchronous), and count, which is a 4-bit output register used to hold the current counter value. The always block is sensitive to the positive edge of both `clk` and `reset`, meaning the logic inside this block will execute whenever the clock signal rises. Inside the module, a condition checks if the reset is active, the counter is immediately set to 1111, setting the count to maximum value. If the reset is not active, the counter decrements by 1 on each rising edge of the clock. Since the counter is 4 bits wide, it can count from 1111 (decimal 15) to 0000 (decimal 0), and then wrap again to 1111, continuing in a loop.

Truth Table:

Clock Pulse	Q3 (MSB)	Q2	Q1	Q0 (LSB)	Binary	Decimal
0	1	1	1	1	1111	15
1	1	1	1	0	1110	14
2	1	1	0	1	1101	13
3	1	1	0	0	1100	12
4	1	0	1	1	1011	11
5	1	0	1	0	1010	10
6	1	0	0	1	1001	9
7	1	0	0	0	1000	8
8	0	1	1	1	0111	7
9	0	1	1	0	0110	6
10	0	1	0	1	0101	5
11	0	1	0	0	0100	4
12	0	0	1	1	0011	3

13	0	0	1	0	0010	2
14	0	0	0	1	0001	1
15	0	0	0	0	0000	0
16	1	1	1	1	1111	15(Repeat)

Application:

These versatile components are crucial in various digital systems, finding applications in areas such as countdown timers, digital displays for time remaining, frequency division (counting down), event countdown tracking, and creating reverse sequences in controllers and automation systems.

Procedure To Create a Lab:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir simple_down_counter`
- 1.4: Open project directory: `cd simple_down_counter`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim down_counter_4bit.v`

RTL Design Code:

```
module down_counter_4bit(
    input clk,
    input reset,
    output reg [3:0] count
);
always @(posedge clk or posedge reset) begin
    if(reset)
        count <= 4'b1111;
    else
        count <= count - 1;
end
endmodule
```

- 2.2: Save the file using command- `:wq!`

Step 3: Create Testbench file

- 3.1: Open gvim editor for testbench with command: `gvim down_counter_4bit_tb.v`

Testbench:

```
module down_counter_tb;
    reg clk;
    reg reset;
    wire [3:0] count;
    down_counter_4bit uut (
        .clk(clk),
        .reset(reset),
        .count(count)
    );
    always #5 clk = ~clk;
    initial begin
        clk = 0;
        $monitor("Time=%t|Reset=%b|count=%b", $time, reset, count);
        reset = 1; #10;
        reset = 0; #200;
        $finish;
    end
    initial begin
        $dumpfile("down_counter_4bit.vcd");
        $dumpvars();
    end
endmodule
```

3.2: Save the testbench file using command- :wq!

3.3: Check the saved file in current directory: use command ls

Step 4: Simulate the Design with Verilator

4.1: Use the following command to simulate:

```
verilator --binary -j 0 -Wall down_counter_4bit.v
down_counter_4bit_tb.v --top down_counter_tb --timing --CFLAGS
"-std=c++20" --trace
```

(Note: type the commands in single line in terminal)

4.2: If any error is available, resolve it.

4.3: After simulation a new directory obj_dir will be created by tool, check it by using command ls.

4.4: Open obj_dir: cd obj_dir

4.5: Use command make -f Vdown_counter_tb.mk Vdown_counter_tb

4.6: A new directory with name Vdown_counter_tb will be created

4.7: Use command ./Vdown_counter_tb to compile

4.8: Check output at terminal

```
Time=      0 | Reset=1 | count=0000
Time=      0 | Reset=1 | count=1111
Time=     10 | Reset=0 | count=1111
Time=     15 | Reset=0 | count=1110
Time=     25 | Reset=0 | count=1101
Time=     35 | Reset=0 | count=1100
Time=     45 | Reset=0 | count=1011
Time=     55 | Reset=0 | count=1010
Time=     65 | Reset=0 | count=1001
Time=     75 | Reset=0 | count=1000
Time=     85 | Reset=0 | count=0111
Time=     95 | Reset=0 | count=0110
Time=    105 | Reset=0 | count=0101
Time=    115 | Reset=0 | count=0100
Time=    125 | Reset=0 | count=0011
Time=    135 | Reset=0 | count=0010
Time=    145 | Reset=0 | count=0001
Time=    155 | Reset=0 | count=0000
Time=    165 | Reset=0 | count=1111
Time=    175 | Reset=0 | count=1110
Time=    185 | Reset=0 | count=1101
Time=    195 | Reset=0 | count=1100
Time=    205 | Reset=0 | count=1011
```

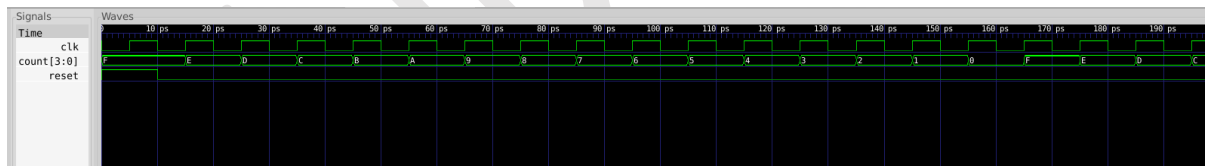
Simulation Output:

The output shown is the simulation result of a 4-bit down counter implemented in Verilog. At the beginning of the simulation, the reset signal is high (Reset=1), which causes the counter to initialize to 1111 (decimal 15). This confirms that the reset is asynchronous, as the counter is immediately set to maximum value without waiting for a clock edge. Once the reset signal goes low (Reset=0), the counter begins decrementing its value on every rising edge of the clock. The count value decreases from 1111 to 0000, representing decimal values from 15 to 0. After reaching 0000, the counter wraps around and starts again from 1111, continuing the countdown sequence. This behavior repeats, demonstrating that the design is functioning correctly as a 4-bit binary down counter with asynchronous reset.

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file:

gtkwave down_counter_4bit.vcd



5.2: Analyze the waveform to verify the correct operation of the 4-bit down counter

Output Waveform:

The output waveform shows the expected behavior of a 4-bit down counter with an asynchronous reset. At the beginning, the reset signal is high, keeping the counter output at 1111 (decimal 15). As soon as the reset goes low, the counter starts decrementing its value on every rising edge of the clock. The count output decreases sequentially from 1111 to 0000, and then wraps around to 1111, continuing the countdown cycle. This confirms that the counter design is functioning correctly, with proper asynchronous reset and 4-bit binary countdown operation.